

Build my OTP-Auth platform for production

RESUME CONTENT

Build my OTP-Auth platform for production using the following exact customer journey and architecture.

CUSTOMER JOURNEY

Customer ↓ Register on optauth.com ↓ Enter website: slotcare.com ↓ DNS verification ↓ Verified ✓ ↓ Choose subdomain: slotcare.otp-auth.com ↓ Subdomain created ✓ ↓ Get Integration ↓ |—— Easy Integration | ↓ | Copy ONE snippet | |—— Developer API ↓ SDK / REST API

1. CUSTOMER REGISTRATION

Create a customer account on optauth.com.

Require email verification before activating the account.

Create a customer/project system so each customer has an isolated project.

Each project should have: - Customer ID - Project ID - Website/domain - Verification status - OTP-Auth subdomain - API credentials - Project status - OTP usage statistics - Rate limits

2. WEBSITE DOMAIN VERIFICATION

Customer enters their real website/domain, for example:

slotcare.com

Generate a unique verification token for that project.

Show the customer a DNS TXT record:

Type: TXT Host: @ Value: otp-auth-verification=UNIQUE_TOKEN

Provide clear instructions for common DNS providers.

When the customer clicks "Verify Domain", the OTP-Auth backend must perform a DNS TXT lookup and verify that the expected token exists.

Do not trust a value supplied by the browser.

Only mark the domain VERIFIED after server-side DNS verification succeeds.

Prevent duplicate/conflicting domain ownership.

3. SUBDOMAIN CREATION

After successful domain verification, allow the customer to choose an OTP-Auth subdomain.

Example:

slotcare.otp-auth.com

Validate: - Allowed characters - Length - Reserved names - Duplicate names - Abuse prevention

The customer must not be able to claim another customer's existing subdomain.

Create and activate the subdomain only after successful domain verification.

4. EASY INTEGRATION — ONE SNIPPET

This must be the default integration for normal customers.

Do NOT require customers to install Composer, install an SDK, use SSH, or run terminal commands.

Provide a copy-paste JavaScript integration such as:

```
<script src="https://cdn.optauth.com/widget.js"></script> <script> OtpAuth.init({ project: "PROJECT_ID" }); </script>
```

The exact implementation can be designed securely.

The browser must never receive a secret API key.

Use only a public project identifier or other appropriate public credential in frontend code.

The widget should support common use cases: - Login OTP - Signup/email verification OTP - Password reset OTP - Email change verification - Other appropriate verification flows

Provide simple methods/components for sending and verifying OTPs.

5. SECURITY OF EASY INTEGRATION

Do not put secret API credentials in JavaScript.

The system must prevent another website from simply copying a customer's project ID and using their OTP service.

Use appropriate server-side validation, allowed-origin/domain controls, rate limiting, abuse detection, and short-lived verification tokens.

The final authentication/session decision should be securely validated by the customer's backend where required.

Do not assume that successful frontend JavaScript execution alone proves authentication.

6. DEVELOPER API

Provide an advanced integration for professional developers.

Expose a documented REST API.

At minimum support:

POST /v1/otp/send POST /v1/otp/verify POST /v1/otp/resend

Design the API with: - Authentication - API key management - Key rotation - Key revocation - Project isolation - Rate limits - Request validation - Idempotency where appropriate - Error codes - Usage tracking

Later support official SDKs for common backend languages such as PHP, Node.js and Python.

The SDK must be optional. REST API integration must always be available.

7. OTP GENERATION

Generate cryptographically secure OTPs.

Do not use predictable random numbers.

Store OTPs securely, preferably as hashes rather than plaintext.

Every OTP must have: - Expiration time - Attempt limit - Purpose - Recipient - Project ID - Creation time - Verification status

Prevent OTP reuse after successful verification.

Prevent brute-force attempts.

8. EMAIL DELIVERY

Use Amazon SES for production email delivery.

The sending identity should use the customer's assigned OTP-Auth subdomain.

Example:

verification@slotcare.otp-auth.com

Configure proper email authentication such as SPF, DKIM and appropriate DMARC configuration.

Do not expose SES credentials to customers.

OTP-Auth owns and manages the SES infrastructure.

Monitor SES sending reputation, bounces, complaints and delivery status.

9. OTP EMAIL SPEED

Optimize the system so that:

Customer website → OTP-Auth API → OTP generation → Amazon SES submission

happens with minimal latency.

The application should not promise an exact inbox-delivery time because recipient mail servers and spam filtering are outside OTP-Auth's control.

Show users a clear message such as:

"OTP sent. It may take a few seconds to arrive."

Provide a secure resend mechanism with cooldown and rate limits.

10. LOGIN/SIGNUP/RESET FLOWS

Design the integration so customers can easily add OTP functionality to:

- Login - Signup - Email verification - Password reset - Email change - Other authentication/verification workflows

The Easy Integration should require minimal coding.

The Advanced API should allow developers to completely control their own UI.

11. CUSTOMER DASHBOARD

Create these sections:

Dashboard Projects Domain Verification Subdomain Integration API Keys OTP Settings Email Templates Usage Logs Security Documentation

Integration page should clearly provide:

Easy Integration → Copy ONE snippet

Advanced Integration → REST API → SDK documentation

Provide copy buttons and examples.

12. TEST MODE

Before production activation, provide a Test Mode.

Customer should be able to:

- Send a test OTP - Receive the test email - Verify the OTP - See API response - Check delivery/log status

Only after the integration passes basic tests should the customer enable Production Mode.

13. SECURITY AND ABUSE PREVENTION

This is an OTP service and must be protected from abuse.

Implement: - Per-project rate limits - Per-recipient rate limits - IP-based protection - OTP attempt limits - Resend cooldown - API key revocation - Suspicious activity detection - Daily/monthly usage limits - Email bounce/complaint monitoring - Project suspension - Audit logs

Never allow an unverified project to send production OTPs.

14. PRODUCTION ARCHITECTURE

Keep the architecture modular so the current development/test hosting can later be migrated to production infrastructure without rewriting the entire application.

Separate: - Frontend - Customer dashboard - Authentication - Domain verification - Subdomain management - OTP service - Email delivery - API - Database - Logging - Rate limiting

15. IMPORTANT DEVELOPMENT RULE

Do not build unnecessary complexity first.

Implement in this order:

1. Customer registration/login 2. Domain DNS verification 3. Subdomain creation 4. Project activation 5. OTP generation 6. SES email delivery 7. OTP verification 8. Easy one-snippet integration 9. Customer dashboard 10. Test mode 11. REST API 12. Advanced SDKs 13. Production security/hardening

At every stage, test the complete flow before moving to the next stage.

The final goal is:

Customer registers → verifies their domain → chooses their OTP-Auth subdomain → gets one-snippet integration → pastes it into their website → their users can request OTPs → OTP-Auth generates and delivers the OTP through SES → users enter the OTP → OTP-Auth verifies it → the customer's application completes the appropriate authentication flow.